

Peptide virtual screening in the pocket of receptor by MolAICal

Qifeng Bai

Email: molaical@yeah.net

Homepage: <https://molaical.github.io> or <https://molaical.gitlab.io>

School of Basic Medical Sciences

Lanzhou University

Lanzhou, Gansu 730000, P. R. China

1. Introduction

This tutorial introduces peptide generation and the steps for peptide virtual screening based on receptor pockets. **LightDock depends on the amino acid types supported by the scoring function. For example, DFIRE only supports the standard amino acid and residue name "MMB", but when using MolAICal's default screening method, it is not limited to specific amino acid types.** This tutorial provides detailed step-by-step methods for peptide virtual screening; if users want to further understand the step-by-step methods for MolAICal to process receptor and ligand structures, please refer to "Quick Start 1" in <https://molaical.github.io>. If users want to learn about more detailed molecular preparation processes, they can study the tutorial: <https://github.com/MolAICal/documents/tree/master/tutorials/002-AIVS>. Here, we use the MolAICal software package for this tutorial.

2. Materials

2.1. Software requirement

- 1) **MolAICal:** <https://molaical.github.io>
- 2) **UCSF Chimera:** <https://www.cgl.ucsf.edu/chimera/download.html>
- 3) **PyMol:** <https://github.com/cgohlke/pymol-open-source-wheels> ,
<https://github.com/maabuu/pymol-wheels> ,
<https://pypi.org/project/pymol-open-source-whl> ,
<https://github.com/cnpem/PyMOL4Win>

2.2. Example files

- 1) All the necessary tutorial files are downloaded from:
https://github.com/MolAICal/tutorials/tree/master/029-peptide_vs_molaical

3. Procedure

3.1 Check materials

Go to the tutorial directory:

```
#> cd 029-peptide_vs_molaical
```

It will see the list of files:

- ◆ **7LLL.pdb:** the complex structure file in pdb format, which is from <https://www.rcsb.org>
- ◆ **P_peptide.pdb:** peptide ligand from the "7LLL.pdb". Preparation methods can be found in

Appendix 1 or the MolAICal protein-peptide docking tutorial.

- ◆ **conf.txt**: the docking parameters that need to be set.
- ◆ **R_protein.pdb**: receptor file from the "7LLL.pdb".

Preparation methods for the above molecular files can be found in Appendix 1 or the MolAICal protein-peptide docking tutorial (see: <https://molaical.github.io/tutorial.html>).

3.2. Prepare docking parameters

1) **Get the box center** of the receptor pocket. Here, there is a peptide bound to the receptor, so the center of this peptide is considered the box center of the receptor pocket. It can get through the commands below:

```
#> molaical.exe -call run -c sfile -i m_center.py P_peptide.pdb
```

It will output:

Geometric center - State 1: 96.382238 109.960593 159.069831

Center of mass - State 1: 96.405446 110.003945 158.961354

Here, **Geometric center** is used. The box size of the receptor pocket can be determined as **Figure 1** based on the obtained geometric center.

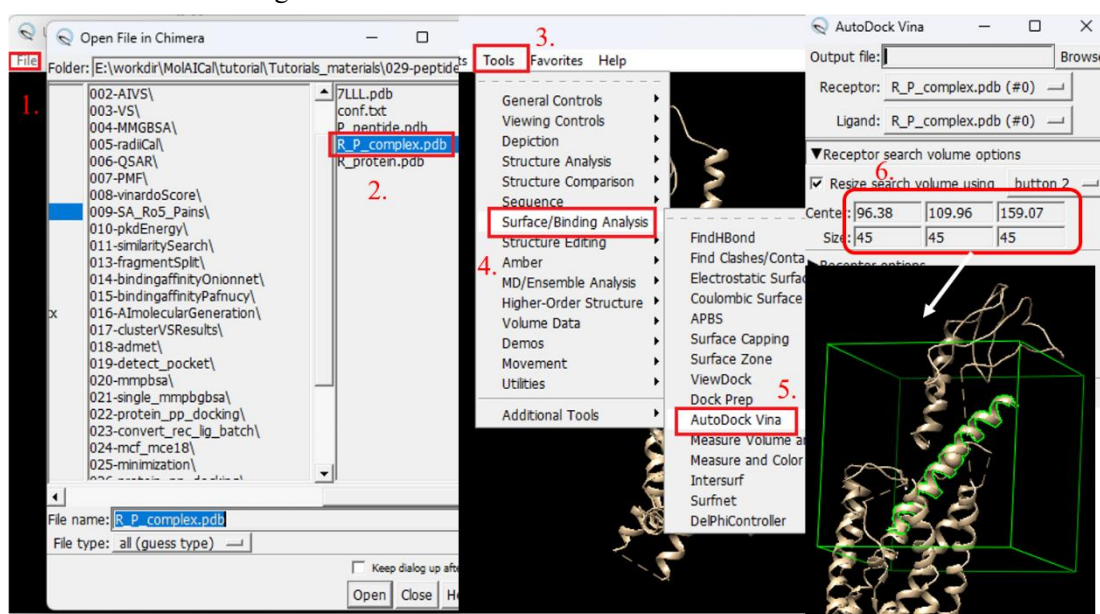


Figure 1

2) **Preparing the receptor file** in PDBQT format based on the command below:

```
#> molaical.exe -dock receptor -i R_protein.pdb
```

It will generate the file "R_protein.pdbqt".

Now update the docking parameter file "**conf.txt**" (**receptor, center, and size**) according to the above results:

```
receptor = R_protein.pdbqt
```

```
cpu = 1
```

```
center_x = 96.38
```

```
center_y = 109.96
```

```
center_z = 159.07
```

```
size_x = 45
```

```
size_y = 45
```

```
size_z = 45
```

```
num_modes = 1
```

3.3. Peptide Generation

Go to the tutorial directory:

```
#> cd 029-peptide_vs_molaical
```

1) Generate the linear peptides by MolAICal

```
#> molaical.exe -call run -c pepgen -i -l 8 -n 2 -3d -nc -o linear_seq.txt -dir linear
```

- ◆ **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- ◆ **-i** : Call the external program. It includes all command-line arguments of external programs after '-i'.
- ◆ **-c (before -i)**: It will generate peptides if its value is "pepgen".
- ◆ **-l**: Number of amino acid residues in the peptide
- ◆ **-n**: Number of unique sequences to generate
- ◆ **-3d**: Generate 3D structures using pyPept
- ◆ **-nc**: Do not add ac/am caps to sequences
- ◆ **-o**: Output file for generated sequences
- ◆ **-dir**: Output directory for 3D structures

For more details, users can refer to the MolAICal manual.

It will see the generated 3D peptides in the folder "linear" which can be used for further virtual screening.

2) Generate the linear peptides by MolAICal with fixed residues, **which can be used for peptide modification.**

```
#> molaical.exe -call run -c pepgen -i -l 8 -n 3 -rc 3 -3d -dir mpep -s EEYVPIST -d none -fix 1 7
```

- ◆ **-dir:** Output directory for 3D structures
- ◆ **-rc:** Number of RDKit random conformers to generate (default: 1), which can give the wanted number of peptides.
- ◆ **-d:** Direction for sequence extension from seed [forward, backward, both, none].
- ◆ **-s:** Starting sequence (optional)
- ◆ **-fix:** Positions (1-indexed) to keep fixed during generation

For more details, users can refer to the MolAICal manual.

3) Generate the cyclic peptides by MolAICal. MolAICal also supports the peptide generation with disulfide bonds among different residues, etc (see MolAICal manual).

```
#> molaical.exe -call run -c pepgen -i -l 8 -n 3 -rc 3 -cc -3d -c 4 -dir cyclic
```

- ◆ **-cc:** Generate cyclic peptides (BILN/HELM only)
- ◆ **-c (after -i):** Number of CPU cores for parallel processing

For more details, users can refer to the MolAICal manual.

In the directories "**linear**", "**mpep**", and "**cyclic**", three types of peptides are included for different purposes and applications. **These peptides will be used for the subsequent virtual screening.** Due to the **time-consuming** nature of peptide virtual screening, **this tutorial only extracts a limited number of peptides for demonstration.** Users can generate sufficient peptides based on their computational resources for research purposes.

4) Filter out the similar molecules: after generating many peptides, some peptides may exhibit highly similar conformations—for instance, those with an RMSD below 2 Å (or a user-defined threshold, e.g., 5 Å) after superposition. To optimize virtual screening efficiency, redundant molecules with conformational similarities below the set threshold can be removed as follows:

```
#> molaical.exe -call run -c sfile -i rmsdcal.py -in . -e .pdb -t 2.0 -a delete -c 8
```

- ◆ **-in:** Input directory. The string "." after "-in" represents the current directory
- ◆ **-e:** File extension, default .pdb; can also be .mol2 or .sdf
- ◆ **-t:** RMSD threshold, default 2.0
- ◆ **-a:** The value can be "keep, delete, or move". "delete" means to delete the latter duplicates
- ◆ **-c (after "-i"):** Number of CPU cores for parallel execution. Default value is the number of all cores.
- ◆ **-c (before "-i"):** Invoking the script file if the value is "sfile".

3.4. Peptide Virtual Screening

Users can select rigid docking or flexible docking for their research.

3.4.1. Virtual screening based on rigid docking

Since MolAICal can generate **multiple conformations of peptide ligands**, **rigid docking** can be performed using MolAICal to **reduce the computational cost**.

```
#> molaical.exe -dock vs -i linear -k pdb -nc 3 -p "-Z"
```

-nc: represents the number of used CPU cores.

-b: switch molecular format conversion. “on” means conversion. “off” means no conversion. The default value is “on”.

-k: the files contain the keyword for virtual screening. The default value is “.mol2”.

-p: can accept additional parameters for format conversion. The value following it must be enclosed in double quotes "". If the value contains multiple parameters, they must be separated by commas ",". The default value is an empty string "". The value "-Z" after the argument '-p' will inactivate all active torsions to realize the rigid docking.

3.4.2. Virtual screening based on flexible docking

For protein and peptide docking, it is very **computationally intensive** and time-consuming. The command is as follows (without adding the '-p' parameter):

```
#> molaical.exe -dock vs -i linear -k pdb -nc 3
```

Note: **Flexible virtual screening is particularly time-consuming**. To obtain screening results **within an acceptable timeframe**, please set a reasonable number of peptides for screening on your local computer or a computing node based on your hardware conditions. For example: **1 CPU core generally corresponds to the screening of approximately 10 peptide molecules**. Then calculate the total number of peptide segments for screening across the entire computing node. Please configure this according to your actual computing capacity.

3.4.3. Continue virtual screening (option)

Should the virtual screening task be terminated prematurely due to unforeseen circumstances, resuming the process requires only two steps:

- 1) Relocate all existing VS result files (containing the default **"_out.pdbqt"** suffix) to another directory by commands:

```
#> molaical.exe -call run -c sfile -i molaical_batch.py -sd linear -ds -ik "_out.pdbqt" > tmplist
```

-ds: Whether search files with appointed characters, if input “-ds” only, it means True to search files with appointed characters.

-sd: The directory for searching.

-ik: The keyword is for searching.

```
#> molaical.exe -call run -c sfile -i molaical_batch.py -cvs -il tmplist -ol l_complete_folder
```

-cvs: Whether prepare for continuing to run virtual screening. If input “-cvs” only, it means True to prepare for continuing to run virtual screening jobs.

-il: The file of ligands list.

-ol: In the section of continuing virtual screening when using “-cvs”, it is folder for storing.

Note: the complete molecules will be moved to the directory “l_complete_folder”.

Option: Sometimes, there may be huge numbers of molecules in one folder. To reduce the number of molecules in one folder, it can assign the number of molecules in one folder as follows:

```
#> molaical.exe -call run -c sfile -i molaical_batch.py -cvs -dp 3 -cvs -il tmplist -ol t1_folder
```

-dp: In the section of continuing virtual screening when using “-cvs”, it is the storing number of screened molecules in every folder. The default value is “None” which means to store all screened molecules in one folder. It can be 1, 2, or 3... which represents the storing number of molecules in every folder.

Note: it will generate the folders “t1_folder1”, “t1_folder2”... each of which contains the storing number after ‘-dp’.

2) Repeat the above virtual screening step to process unscreened molecules.

4. Results

4.1 Checking results

Users can use Pymol software (<https://github.com/cgohlke/pymol-open-source-wheels> or <https://www.cgohlke.com>) to load ligands with PDBQT format directly. Here, Pymol is introduced to show docking results.

[Open 029-peptide_vs_molaical/linear](#)

Use Pymol to load a case “LKGIWWPL_alpha-helix_out.pdbqt” and “R_protein.pdb”. The docking result is shown in Figure 2, which indicates MolAICal obtains a suitable peptide ligand. If users want to analyze the hydrogen bonding interaction between the docked peptide and protein, the hydrogen atoms should be added to the docked peptide and protein using Pymol.

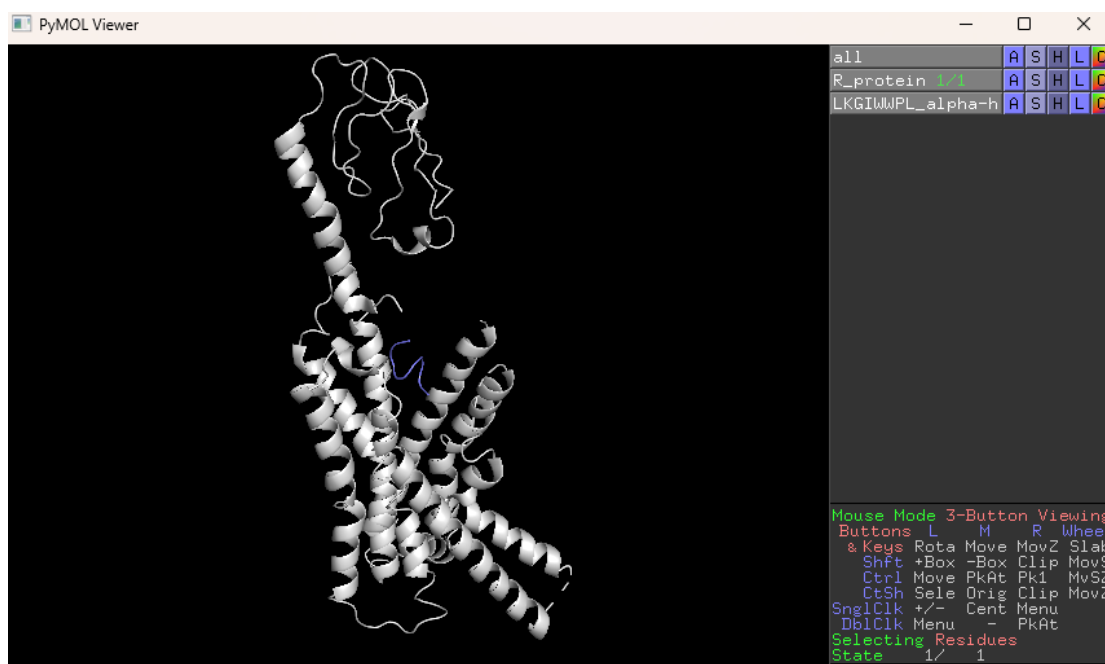


Figure 2

4.2 Ranking virtual screening results

Go to folder "029-peptide_vs_molaical", and run the following command:

```
#> molaical.exe -call run -c sfile -i printscore.py "linear/*out.pdbqt"
```

Here, "linear" is the directory containing the virtual screening results. If a restart task has been executed, switch to the directory generated after the restart task (e.g., the "l_complete_folder" directory in this specific task). It will show the ligand name and corresponding binding score (Sorted in ascending order by score). If users want to save results into a file, they can use the following command:

```
#> molaical.exe -call run -c sfile -i printscore.py "linear/*out.pdbqt" > results.log
```

Note: "molaical.exe" is your installed directory path of MolaICaI. All the useful scripts are supplied in the folder "scripts" of MolaICaI.

4.3 Extract the top-ranked molecule into the new folder

If users want to move top-ranked molecules into a new folder, for example, in this tutorial, if users want to move 2 top-ranked molecules into a new folder named "results", they can use the command as follows:

```
#> molaical.exe -call run -c sfile -i get_top_results.py "linear/*out.pdbqt" 2 results
```

Two top-ranked molecules will be moved into the folder "results".

5. (Option) Linux Version of MolAICal: Architecture and Acceleration

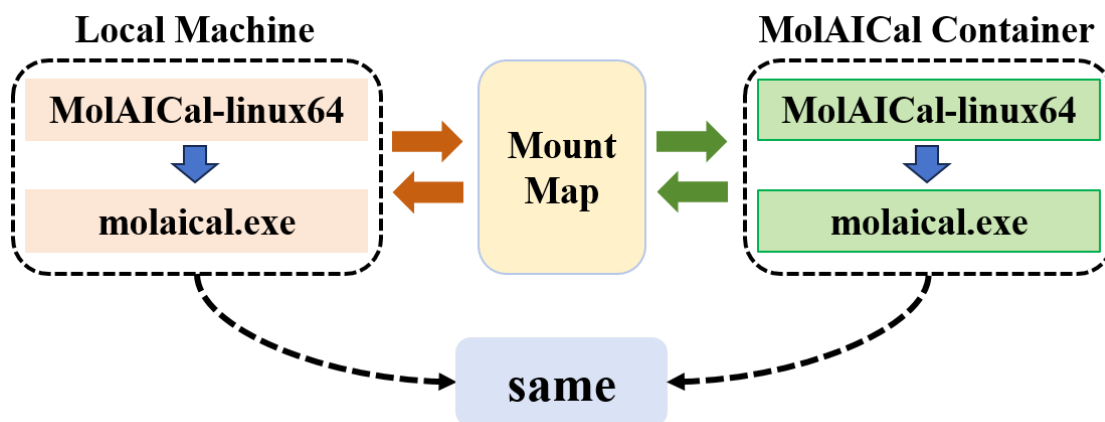


Figure 3. Architecture Diagram for the Linux Version of MolAICal

As shown in Figure 3, the **MolAICal container** can not only utilize the container system's files and programs but also access the host machine's library files through mounting and mapping mechanisms. This effectively **resolves** issues such as **library conflicts** or missing dependencies. However, **program execution within containers may indeed be slower than on the local machine.**

Figure 3 demonstrates that due to the mapping relationship, the computational commands executed by "**molaical.exe**" in the container are **identical** to those run by "**molaical.exe**" on the local machine.

◆ Therefore, **for tasks with minimal library dependencies**—such as virtual screening using MolAICal—**consider copying MolAICal from the container to the local machine:**

```
***** 1. Go to the container file system, it will enter the "/root" *****
#> molaical.exe -cset shell in

*** 2. The first part of 'cp' is from the MolAICal container, and the second part is in the local machine ***
#> cp -r soft/MolAICal-linux64 <local machine directory>/

***** 3. Exit container file system *****
#> exit
```

◆ At last, performing virtual screening locally via the same methodology can significantly accelerate the screening process.

Appendix 1

Appendix 1 below **describes** the method for **preparing the materials for this tutorial**, which follows the **same** procedure as **protein-peptide molecular docking**; if you are already familiar with the protein-peptide molecular docking workflow, this section can be skipped.

1. Deal with molecular files

Load PDB file 7LLL.pdb into PyMol, and use the steps of **Figure a1** to save "[R_P_complex.pdb](#)", "[R_protein.pdb](#)", and "[P_peptide.pdb](#)", respectively.

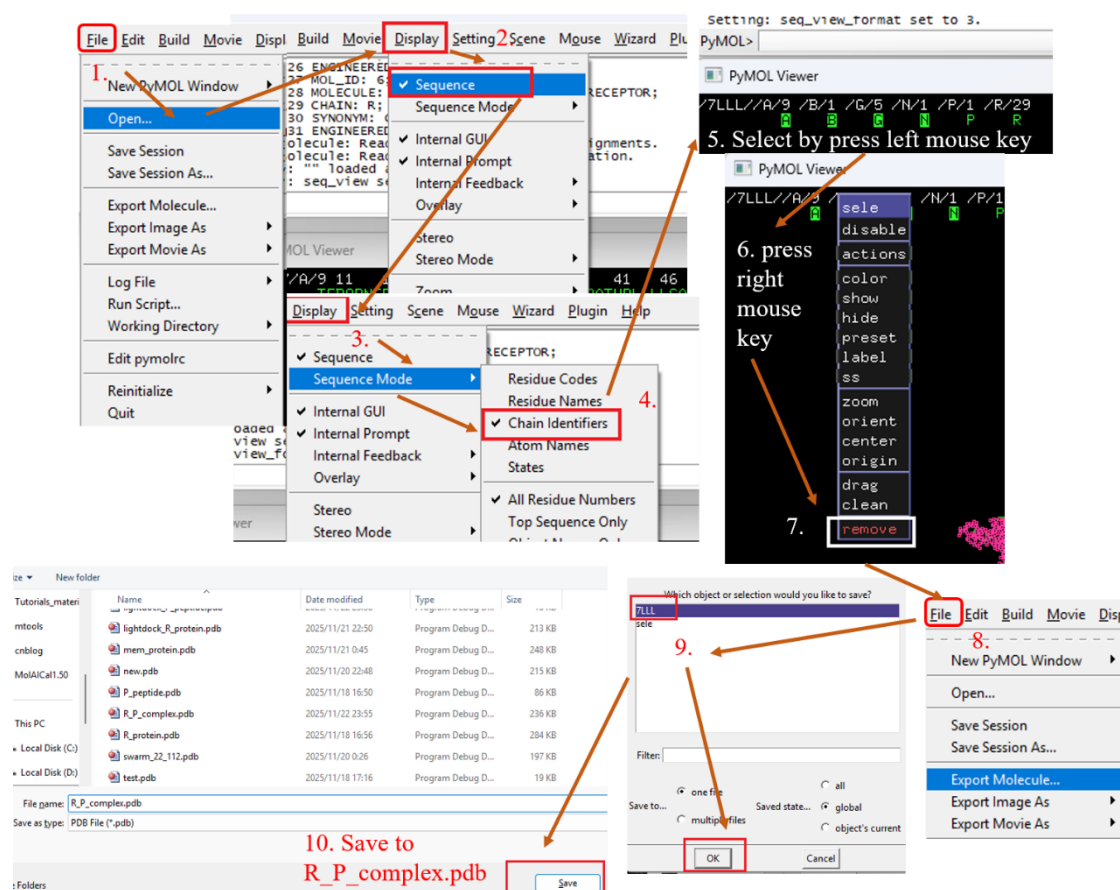


Figure a1

The files "[R_P_complex.pdb](#)" (chain name is "P" and "R"), "[R_protein.pdb](#)" (chain name is "R"), and "[P_peptide.pdb](#)" (chain name is "P") are extracted from the complex of the glucagon-like peptide-1 receptor (GLP-1R) and exendin-4, the first FDA-approved GLP-1R agonist (PDB ID: 7LLL.pdb). The agonist exendin-4 is a peptide. If users want to **perform protein-protein or protein-nucleic acids virtual screening**, please substitute the peptide file with the protein or nucleic file because the peptide can be considered a small protein.